LABORATORIO 10 · INTERNET DE LAS COSAS

ESTACIÓN METEOROLÓGICA IoT

CC3067 · UVG · TOPIC 23775

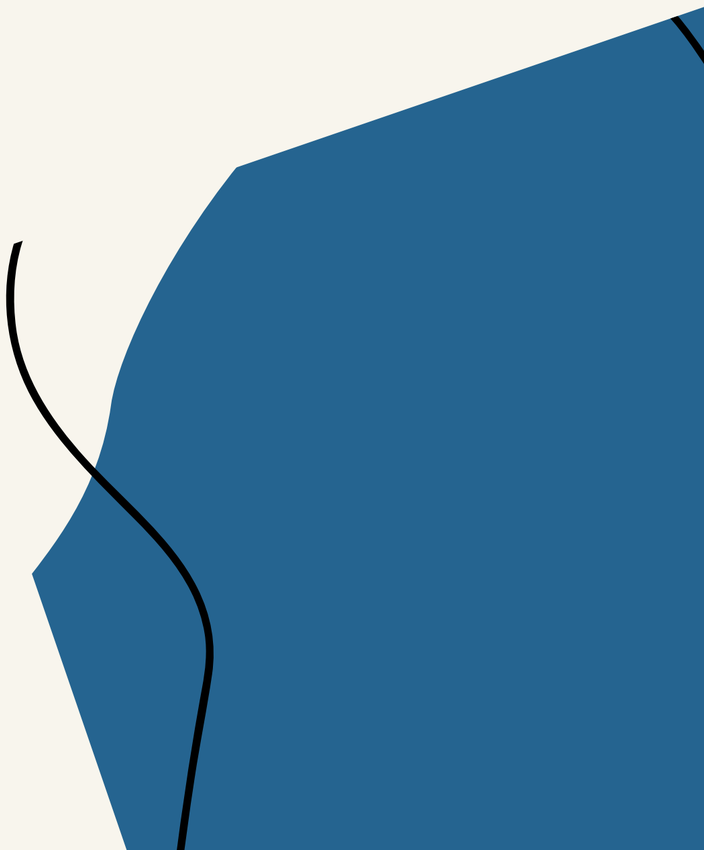
¿QUÉ IMPLEMENTAMOS?

Simulamos una estación meteorológica con tres sensores.

Un Producer publica cada lectura en Kafka.

Un Consumer la recibe y actualiza las gráficas.

Sensores → Producer → Kafka Edge → Consumer → Gráficas



SENSORES SIMULADOS

Temperatura

0 a 110.00 °C
Dos decimales

Humedad relativa

0 a 100 %
Valor entero

Dirección del viento

N, NO, O, SO, S, SE, E, NE
8 categorías

Distribución normal

Media 55
Desviación estándar 15

JSON Y CAPA OSI

{"temperatura": 66.69, "humedad": 49,
"direccion_viento": "SE"}

¿A qué capa pertenecen?

Capa de aplicación (7). JSON representa datos y SOAP define mensajes entre aplicaciones.

¿Qué ventajas ofrecen?

Estructuran la telemetría, facilitan el intercambio entre lenguajes y permiten agregar campos.

ENVÍO DE DATOS A KAFKA

Producer implementado en Python (kafka-python).

Servidor Edge: 161.35.11.141:9092

Topic: 23775 Key: sensor1

Autenticación: SASL_PLAINTEXT / PLAIN

Cada lectura se envía entre 15 y 30 segundos.

CONSUMO Y VISUALIZACIÓN

KafkaConsumer se suscribe al mismo topic.

Lee mensajes anteriores y los que llegan nuevos.

Reconoce JSON y el formato binario de 3 bytes.

Actualiza temperatura, humedad y viento al recibir datos.

EVIDENCIA DEL PRODUCTOR

```
(venv) PS C:\Users\miafu\Downloads\CC3067-internet_of_things> python producer.py --compacto
C:\Users\miafu\Downloads\CC3067-internet_of_things\producer.py:74: DeprecationWarning: key_serializer does not implement kaf
ka.serializer.Serializer
  producer = crear_producer(args.compacto)
C:\Users\miafu\Downloads\CC3067-internet_of_things\producer.py:74: DeprecationWarning: value_serializer does not implement k
afka.serializer.Serializer
  producer = crear_producer(args.compacto)
Conectado a 161.35.11.141:9092. Publicando en el topic '23775'.
Formato: compacto (3 bytes).
Ctrl+C para detener.

[20:55:34] #1 enviado -> {"temperatura": 66.69, "humedad": 49, "direccion_viento": "SE"} (particion 1, offset 0)
[20:55:59] #2 enviado -> {"temperatura": 53.49, "humedad": 81, "direccion_viento": "NE"} (particion 1, offset 1)
[20:56:17] #3 enviado -> {"temperatura": 21.16, "humedad": 85, "direccion_viento": "SE"} (particion 1, offset 2)
[20:56:40] #4 enviado -> {"temperatura": 39.75, "humedad": 82, "direccion_viento": "O"} (particion 1, offset 3)
[20:57:00] #5 enviado -> {"temperatura": 88.47, "humedad": 49, "direccion_viento": "O"} (particion 1, offset 4)
[20:57:23] #6 enviado -> {"temperatura": 62.72, "humedad": 55, "direccion_viento": "N"} (particion 1, offset 5)
[20:57:52] #7 enviado -> {"temperatura": 40.08, "humedad": 74, "direccion_viento": "NE"} (particion 1, offset 6)
[20:58:13] #8 enviado -> {"temperatura": 35.94, "humedad": 30, "direccion_viento": "N"} (particion 1, offset 7)
[20:58:36] #9 enviado -> {"temperatura": 37.71, "humedad": 45, "direccion_viento": "NO"} (particion 1, offset 8)
[20:58:54] #10 enviado -> {"temperatura": 68.96, "humedad": 50, "direccion_viento": "E"} (particion 1, offset 9)

Interrumpido. Lecturas enviadas: 10
```

10 mensajes enviados en modo compacto. Confirmación de offsets 0 a 9.

EVIDENCIA DEL CONSUMIDOR

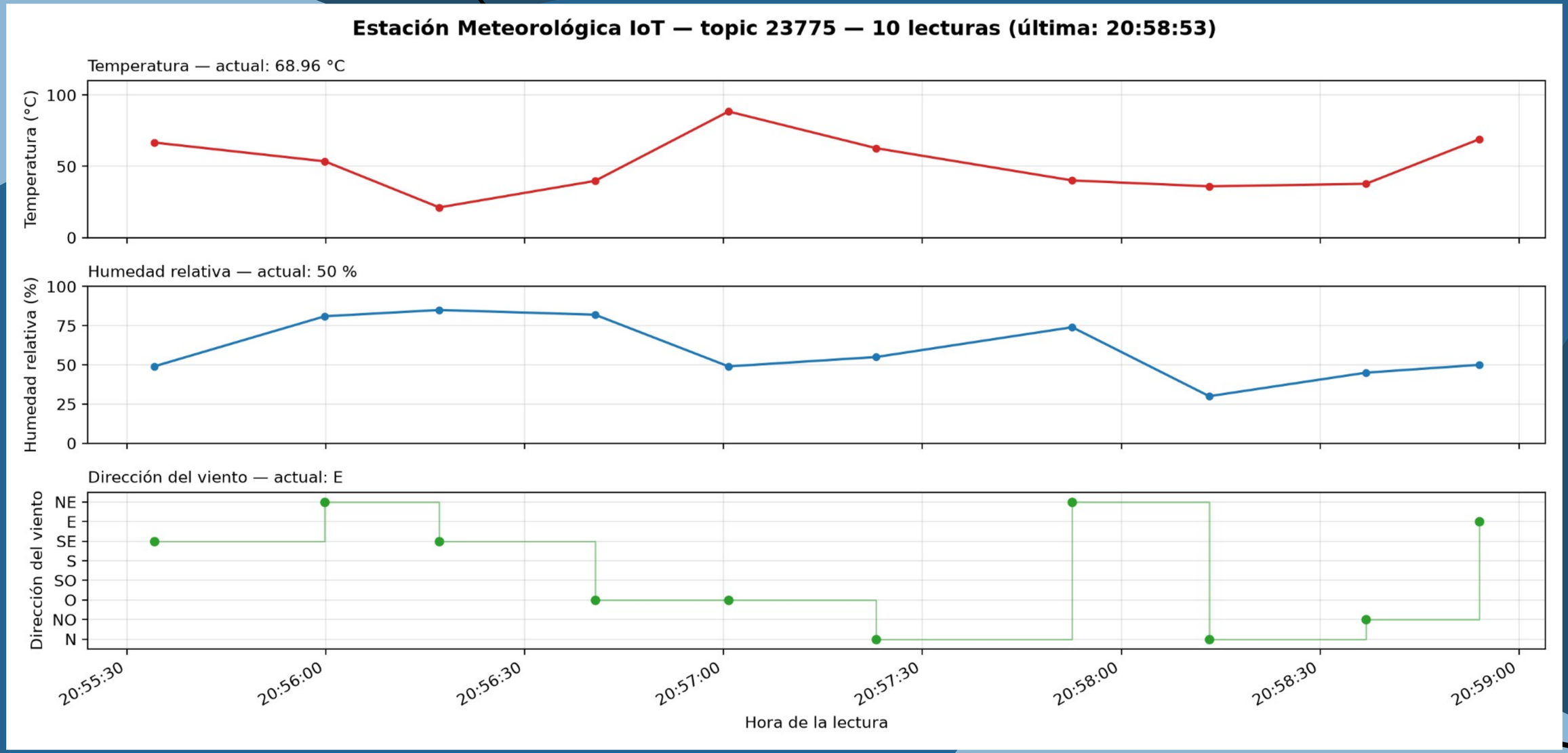
```
(venv) PS C:\Users\miafu\Downloads\CC3067-internet_of_things> python consumer.py
Conectado a 161.35.11.141:9092. Escuchando el topic '23775'.
Ctrl+C o cerrar la ventana para detener.

[20:55:34] offset 0 -> {'temperatura': 66.69, 'humedad': 49, 'direccion_viento': 'SE'}
[20:55:59] offset 1 -> {'temperatura': 53.49, 'humedad': 81, 'direccion_viento': 'NE'}
[20:56:17] offset 2 -> {'temperatura': 21.16, 'humedad': 85, 'direccion_viento': 'SE'}
[20:56:40] offset 3 -> {'temperatura': 39.75, 'humedad': 82, 'direccion_viento': 'O'}
[20:57:00] offset 4 -> {'temperatura': 88.47, 'humedad': 49, 'direccion_viento': 'O'}
[20:57:23] offset 5 -> {'temperatura': 62.72, 'humedad': 55, 'direccion_viento': 'N'}
[20:57:52] offset 6 -> {'temperatura': 40.08, 'humedad': 74, 'direccion_viento': 'NE'}
[20:58:13] offset 7 -> {'temperatura': 35.94, 'humedad': 30, 'direccion_viento': 'N'}
[20:58:36] offset 8 -> {'temperatura': 37.71, 'humedad': 45, 'direccion_viento': 'NO'}
[20:58:53] offset 9 -> {'temperatura': 68.96, 'humedad': 50, 'direccion_viento': 'E'}

Consumer detenido. Lecturas graficadas: 10
Grafica guardada en grafica_telemetria.png
(venv) PS C:\Users\miafu\Downloads\CC3067-internet_of_things>
```

El Consumer recibió los mismos valores y guardó la imagen de la gráfica.

TELEMETRÍA EN TIEMPO REAL



RESULTADOS OBSERVADOS

10

lecturas recibidas

0 a 9

offsets coincidentes

18 a 29 s

intervalo observado

3 bytes

payload compacto

Temperatura: 21.16 a 88.47 °C (media: 51.50 °C)
Humedad: 30 a 85 % (media: 60 %)
Se observaron 6 direcciones distintas del viento.

VENTAJAS Y DESVENTAJAS DE PUB/SUB

Ventajas

Producer y Consumer funcionan de forma independiente.

El broker conserva mensajes para consumirlos después.

Permite incorporar más consumidores.

Desventajas

Se debe instalar y mantener el broker.

La configuración y la autenticación agregan complejidad.

Una falla del servicio puede retrasar la entrega.

¿CUÁNDO CONVIENE USAR KAFKA?

Sí tiene sentido

Telemetría IoT continua

Registros de aplicaciones

Flujos de eventos con varios consumidores

No suele hacer falta

Una aplicación muy pequeña

Consultas puntuales

Comunicación síncrona directa entre dos componentes

RESTRICCIÓN DE 3 BYTES

14 bits

Temperatura

0 a 11000

°C × 100

7 bits

Humedad

0 a 100

Entero

3 bits

Viento

0 a 7

8 direcciones

14 + 7 + 3 = 24 bits. La temperatura máxima codificada es 11000, menor que 2^{14} = 16384.
El límite exige fijar rangos, precisión y significado de cada bit.

CODIFICACIÓN Y DECODIFICACIÓN

Encode

```
t = round(temperatura * 100)
valor = (t << 10) | (humedad << 3) | viento
payload = valor.to_bytes(3, 'big')
```

Decode

```
n = int.from_bytes(payload, 'big')
temperatura = (n >> 10) / 100
humedad = (n >> 3) & 0x7F
viento = direcciones[n & 0x07]
```

Ejemplo calculado: 66.69 °C, 49 %, SE = 68 35 8D (hexadecimal), 3 bytes.

¿Y SI LA HUMEDAD TUVIERA UN DECIMAL?

De 0.0 a 100.0 % hay 1001 valores posibles.

La humedad necesitaría 10 bits en lugar de 7.

14 + 10 + 3 = 27 bits. El mensaje ya no cabría en 3 bytes.

Para conservar el límite, podríamos reducir la temperatura a 0.1 °C: 11 + 10 + 3 = 24 bits.

KAFKA ANTE MAYORES RESTRICCIONES

batch_size y linger_ms agrupan registros antes de enviarlos.

compression_type puede reducir el tráfico de lotes.

Los reintentos ayudan a recuperar envíos tras fallas temporales.

Estas opciones reducen el costo de transporte, pero no reemplazan la codificación cuando cada payload tiene un límite estricto de 3 bytes.

CONCLUSIONES

Comunicación real

Kafka confirmó 10 envíos y el Consumer recibió las mismas 10 lecturas.

Formato compacto

La distribución de 14/7/3 bits conserva los valores en tres bytes.

Gráficas

La temperatura, humedad y dirección del viento se actualizaron con los mensajes recibidos.

Compromiso de precisión

Agregar decimales exige más bits o reducir la resolución de otro sensor.

GRACIAS

POR LA ATENCIÓN

ESTACIÓN METEOROLÓGICA IoT · CC3067